EXPENSE ANALYZER

A Python-Based Personal Expense Tracking and Data Analysis System

PROJECT REPORT

Submitted by : Santhipriyan R – 2025BCS0033

Course: ICS 214 – IT Workshop III

Project Area: Python Programming and Data Analytics

Academic Project

Abstract

Expense Analyzer is a Python-based application designed to record, organize, manage, and analyse personal expenses. The project combines a command-line interface, a Flask-based web interface, CSV files, an SQLite database, and a Jupyter Notebook for data analysis. NumPy and Pandas support numerical calculations and tabular analysis, while Matplotlib and Seaborn support visual exploration of spending patterns. The project also includes monthly budget functionality and exportable summary reports.

During verification, the current dataset contained 10 unique transactions with a total recorded expenditure of ₹2,649.00. The CSV and SQLite records were compared and matched exactly. The source files passed syntax checks, the notebook was confirmed to be valid JSON with 22 cells, and the generated PDF report was checked for required sections.

1. Problem Statement

Personal expenses can be difficult to track when transaction details are scattered across notes or separate files. Without organized records and summaries, it is harder to understand where money is being spent, compare spending across months, or check expenses against a budget. This project addresses that problem by providing a small, locally runnable system for recording expenses and producing summaries and visualizations.

2. Objectives

- Record expenses with details such as date, category, description, and amount.
- Support viewing, searching, editing, and deleting expense records.
- Store and retrieve structured records using CSV and SQLite.
- Analyse expenses by category and month using Pandas and NumPy.
- Visualize spending patterns using charts.
- Support monthly budget entry and budget checks.
- Provide a web interface using Flask and a notebook for analysis.
- Generate summary outputs and document the verification process.

3. Technologies Used

Technology	Purpose
Python	Application logic, validation, and integration.
NumPy	Numerical operations and calculations.
Pandas	Data loading, filtering, grouping, and summaries.
Matplotlib	Data visualizations.
Seaborn	Statistical chart styling and visual exploration in the notebook.
SQLite	Local relational database for expense records.
Flask	Web application and routes.
Jupyter Notebook	Interactive analysis and presentation of results.
ReportLab / python-docx	PDF report generation and editable Word report preparation.

4. System Design and Methodology

The project is organized into separate components so that data storage, application operations, the web interface, and analytical work can be handled independently.

Component	Responsibility
expense_analyzer.py	Command-line interface, input handling, expense operations, analysis, charts, and exports.
expense_database.py	SQLite database initialization and database operations.
main.py	Flask web application and HTTP routes.
Expense_Analyzer.ipynb	Notebook-based data analysis and visualizations.
expenses.csv	CSV representation of expense transactions.
expenses.db	SQLite database file containing the expense table.
budget.csv	Monthly budget data.
reports/	Generated charts, summary CSV files, and report documents.

4.1 Processing workflow

1. Load expense records from the selected data source.
2. Validate user input such as dates, amounts, and record identifiers.
3. Perform the requested operation, such as viewing, searching, adding, editing, or deleting an expense.
4. Persist changes through the application's storage functions.
5. Use Pandas and NumPy to calculate summaries and inspect spending patterns.
6. Generate charts and summary files for interpretation and reporting.

5. Main Features

- Expense management: add, view, search, edit, and delete transactions.
- Input validation for required fields and positive expense amounts.
- SQLite storage with database operations separated into a dedicated module.
- Monthly budget storage and budget-check functionality.
- Category and monthly expense analysis.
- Charts for category distribution and spending trends.
- CSV summary exports for analytical results.
- Flask web interface for interacting with the application.
- Jupyter Notebook for analysis, verification, and visualization.

6. Data Analysis Results

The following figures describe the dataset used during the final verification checks. They should be updated if the underlying project data changes before submission.

Measure	Verified result
Number of transactions	10
Unique transaction IDs	10
Total expenditure	₹2,649.00
Average expenditure per transaction	₹264.90
Number of categories represented	7
Date range	20 March 2026 – 18 October 2026

6.1 Category totals

Category	Total expenditure
Food	₹630.00
Shopping	₹550.00
Other categories	Combined remainder of the verified total

Food and Shopping were the two largest category totals in the verified dataset. The other five represented categories account for the remaining expenditure.

6.2 Monthly totals

Month	Total expenditure
March 2026	₹400.00
August 2026	₹450.00
September 2026	₹1,079.00
October 2026	₹720.00

September 2026 had the highest monthly expenditure among the months represented in the verified data. Monthly comparisons can help identify periods with relatively high spending.

7. Testing and Verification

Verification item	Result
CSV and SQLite record comparison	Passed: 10 unique transactions matched exactly.
Total comparison	Passed: both sources totalled ₹2,649.00.
Source syntax checks	Passed for expense_analyzer.py, expense_database.py, and main.py.
Notebook integrity	Passed: valid notebook JSON with 22 cells.
CRUD and validation tests	Previously passed using the project test workflow.
Flask route tests	Previously passed using temporary files.
Chart generation and summary exports	Previously passed; generated outputs preserved.
PDF content verification	Passed: 7 pages and required report sections found.

Tests involving temporary files were used where appropriate to avoid altering the original project data. The final verification also confirmed the presence of the main source files, notebook, data files, README, and final PDF.

8. Limitations

- The project is designed primarily for local use rather than multi-user cloud deployment.
- It does not automatically import transactions from banks.
- The usefulness of the analysis depends on the accuracy and completeness of entered records.
- The current dataset is small and is intended to demonstrate the application's functionality rather than represent a large-scale financial dataset.

9. Future Scope

- Add user authentication and role-based access if multi-user support is required.
- Provide secure backup and restore options.
- Add configurable date-range filters and more comparison charts.

- Improve the interface for mobile and accessibility use.
- Support importing transactions from additional file formats.

10. Conclusion

Expense Analyzer demonstrates how Python programming can be combined with data analysis, visualization, database integration, and a web interface to address a practical personal expense tracking problem. The application supports core expense-management operations, monthly budget functionality, and analytical summaries. Verification confirmed that the CSV and SQLite data matched for the current 10 transactions, with a total expenditure of ₹2,649.00. The project also includes a valid Jupyter Notebook, tested source files, generated reports, and documentation. It provides a foundation for further improvements while remaining suitable for explaining the main concepts during a project demonstration and viva.

Appendix A. How to Run the Project

Run these commands from the project directory in Windows PowerShell.

Terminal application: .\.venv\Scripts\python.exe .\expense_analyzer.py

Flask web application: .\.venv\Scripts\python.exe .\main.py

Jupyter Notebook: .\.venv\Scripts\python.exe -m notebook

The virtual environment must be created and dependencies installed on the computer before these commands will work. The .venv folder itself is usually excluded from the submission ZIP; the README should describe the setup steps.

Appendix B. Project Files

File / folder	Description
expense_analyzer.py	Terminal application.
expense_database.py	SQLite database layer.
main.py	Flask web application.
Expense_Analyzer.ipynb	Analysis notebook.
expenses.csv / expenses.db	Expense data sources.
budget.csv	Monthly budget data.
README.md	Setup, usage, and project notes.
reports/	Charts, exported summaries, and final report.